Relatório Projeto ASA 26/27

Grupo: AL048
Aluno(s): Manuel André (ist1118928) e Bruno Santos (ist1118057)

Descrição do Problema e da Solução

Dados *n* tipos de blocos em forma de paralelepípedo, pretende-se construir uma torre de altura máxima. Cada bloco pode ser rodado, mas as duas dimensões da base de um bloco superior têm de ser estritamente menores do que as do bloco imediatamente inferior. A saída contém a altura, o número de blocos e as suas dimensões, da base ao topo. O modelo tratado permite reutilizar um tipo de bloco em orientações distintas.

A solução foi implementada em **C++**. Cada tipo gera três orientações, uma por cada dimensão usada como altura. Cada orientação é representada por (a, b, h), onde a e b são a largura e o comprimento da base, ordenados de forma a que $a \leq b$, e h é a altura. As orientações são ordenadas por a decrescente e, nos empates, por b crescente, e o problema reduz-se a uma variante da maior subsequência crescente resolvida por programação dinâmica: para cada orientação guarda-se a melhor torre que a tem no topo. Uma primeira versão procurava essa torre percorrendo todas as orientações anteriores, com custo $O(N^2)$, o que se revelou inviável para instâncias grandes. A versão final usa uma **Fenwick Tree** de máximos sobre as coordenadas comprimidas de b, que encontra a melhor torre compatível em $O(\log N)$; os ponteiros para o estado anterior permitem reconstruir a solução.

Análise Teórica

Seja N o número de tipos de blocos. Cada tipo gera 3 orientações, logo trabalhamos com $3N$ orientações e $O(3N) = O(N)$. Seja $D[i]$ a altura máxima de uma torre com a orientação i no topo:

$$D[i] = h_i + \max \{ D[k] : k < i, a_k > a_i, b_k > b_i \}, \quad \text{com } \max \varnothing = 0$$

Como a compressão de b é feita por ordem decrescente, consultar a Fenwick Tree até à posição $p - 1$ (imediatamente antes da posição da orientação atual) considera apenas valores de b estritamente maiores. Nos empates de a, os estados já processados têm b menor ou igual e ficam excluídos da consulta. Assim, a consulta devolve a torre mais alta sobre a qual o bloco atual pode ser colocado, e o máximo de D sobre todas as orientações é a solução ótima.

- **Leitura dos dados** e geração das 3 orientações de cada bloco (ordenar 3 valores é $O(1)$): ciclo linear em N. Logo, **$O(N)$**.
- **Ordenação** das orientações por a decrescente e b crescente. Logo, **$O(N \log N)$**.
- **Compressão de coordenadas** de b (ordenação e remoção de duplicados) e inicialização da Fenwick Tree. Logo, **$O(N \log N)$**.

- **Programação dinâmica:** para cada orientação, pesquisa binária da posição de *b*, consulta e atualização da Fenwick Tree, cada uma O(log N), e cálculo do estado em O(1). Logo, **O(N log N)**.
- **Reconstrução** da torre pelos ponteiros para o estado anterior. Logo, **O(N)**.
- **Apresentação dos dados:** impressão da torre da base para o topo. Logo, **O(N)**.

Complexidade global da solução: **O(N log N)** em tempo e **O(N)** em memória (orientações, estados, coordenadas comprimidas, Fenwick Tree e torre reconstruída).

Pseudocódigo

```
TorreMaxima(blocos[1..N])
 let O = três orientações por bloco, com bases a ≤ b              ▷ O(N)
 ordenar O por a decrescente e, nos empates, b crescente          ▷ O(N log N)
 let V = valores distintos de b, por ordem decrescente            ▷ O(N log N)
 let m = 3N,  q = |V|
 let D[0..m], anterior[0..m], contagem[0..m] be new vectors
 let BIT[1..q] be a new vector initialized to 0
 D[0] = 0;  contagem[0] = 0;  melhor = 0
 for i = 1 to m do                                                ▷ total: O(N log N)
     p = posição de O[i].b em V, por pesquisa binária             ▷ O(log N)
     k = BIT.consultar(p − 1, D)                                  ▷ O(log N)
     D[i] = O[i].h + D[k]
     anterior[i] = k
     contagem[i] = 1 + contagem[k]
     BIT.atualizar(p, i, D)                                       ▷ O(log N)
     if D[i] > D[melhor] then
         melhor = i
     endif
 endfor
 let torre be an empty list
 k = melhor
 while k ≠ 0 do                                                   ▷ O(N)
     acrescentar O[k] ao fim de torre
     k = anterior[k]
 endwhile
 inverter torre                                                   ▷ O(N)
 return D[melhor], contagem[melhor], torre
```

Avaliação Experimental

Todos os testes foram executados num computador portátil com 8 GB de memória, CPU Intel® Core™ i7-7500U (4 threads) a 3,50 GHz, com Ubuntu 24.04.5 LTS. O programa foi compilado com `g++ -std=c++11 -O3`.

Foram geradas 20 instâncias para cada algoritmo com o gerador fornecido (`./gerador N 1000 42`: dimensão máxima 1000 e *seed* fixa), variando apenas o número de blocos N. O tempo de CPU (*user*) e a memória máxima residente foram medidos com `/usr/bin/time`. Para o algoritmo 1 (versão final, O(N log N)) indica-se a mediana de 3 execuções; para o algoritmo 2 (versão inicial, $O(N^2)$) uma única execução, dado o tempo de cada uma. O algoritmo 2 foi testado apenas até N = 100 000, pois com N = 50 000 já demora 33 s.

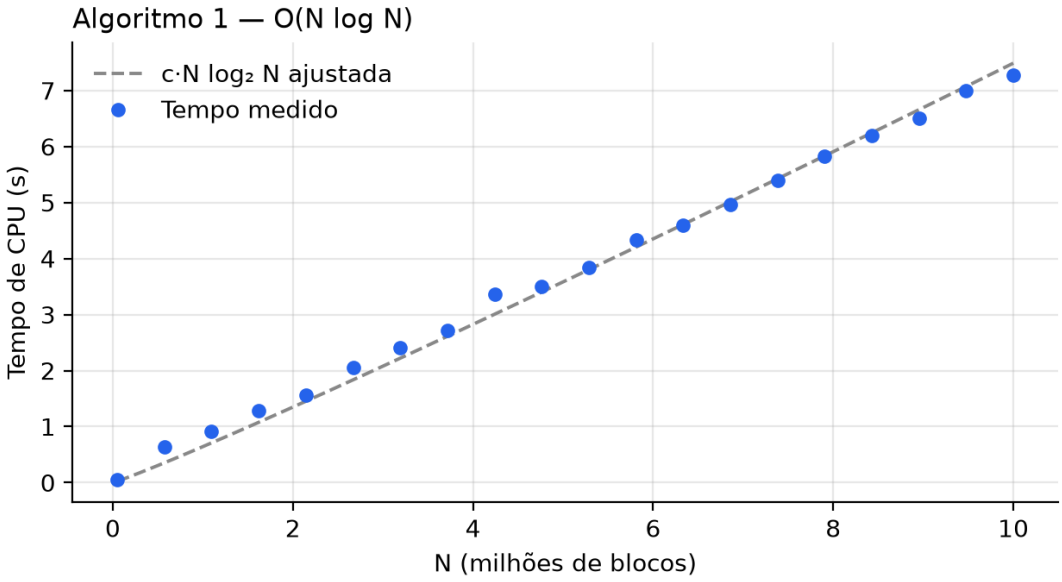


Figura 1 — Tempo de CPU do algoritmo 1 em função de N, com a curva c·N log₂N ajustada aos dados (mínimos quadrados).

Os pontos medidos acompanham a curva c·N log₂N ajustada ao longo de todo o intervalo, até 10 milhões de blocos. Dividindo o tempo por N log₂N obtém-se um valor praticamente constante, cerca de 32 ns a partir de 2 milhões de blocos (para N pequeno é mais alto devido a custos fixos, como o arranque do processo). A memória máxima cresce linearmente, de 9 MB para 50 000 blocos até 1,1 GB para 10 milhões, cerca de 115 bytes por bloco. Isto confirma a complexidade O(N log N) em tempo e O(N) em memória prevista na análise teórica.

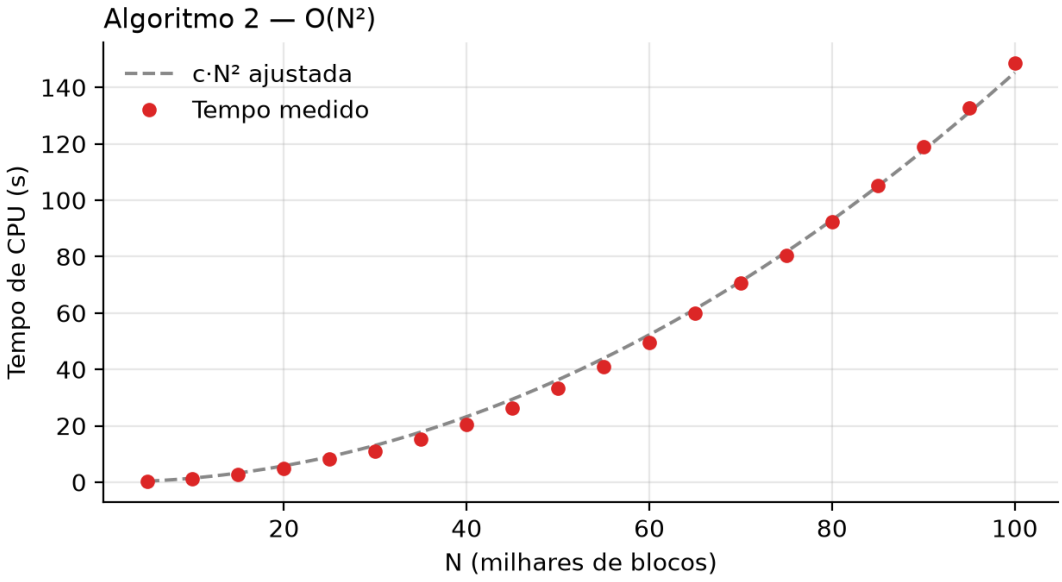


Figura 2 — Tempo de CPU do algoritmo 2 em função de N, com a curva c·N² ajustada aos dados (mínimos quadrados).

O algoritmo 2 apresenta uma curva claramente parabólica, bem descrita por c·N². O quociente T / N² mantém-se entre 12 e 15 ns, e duplicar N quadruplica o tempo (33,4 s para N = 50 000 e 148,6 s para N = 100 000), o que confirma o comportamento

quadrático. A memória continua linear (3,6 MB a 13,4 MB) e até é inferior à do algoritmo 1: a diferença entre as duas versões está no tempo, não no espaço.

Estimativa para N = 10 000 000

Recorrendo a matemática simples, podemos estimar quanto tempo levaria o algoritmo 2 com 10 000 000 de blocos, o maior valor testado no algoritmo 1. Como o tempo é proporcional a N^2, multiplicar N por 100 multiplica o tempo por $100^2 = 10\,000$:

$$T_2(10\,000\,000) \approx T_2(100\,000) \times (10\,000\,000 / 100\,000)^2$$
$$= 148{,}59 \text{ s} \times 10\,000$$
$$= 1\,485\,900 \text{ s} \approx 412{,}8 \text{ horas} \approx \mathbf{17{,}2 \text{ dias}}$$

O algoritmo 1 resolve a mesma instância em 7,28 s, ou seja, cerca de **204 000 vezes** mais depressa (1 485 900 / 7,28). Este resultado mostra o impacto prático de passar de $O(N^2)$ para $O(N \log N)$ com a Fenwick Tree: o algoritmo 2 seria inutilizável para instâncias desta dimensão, enquanto o algoritmo 1 as resolve em poucos segundos.

Comparação da memória

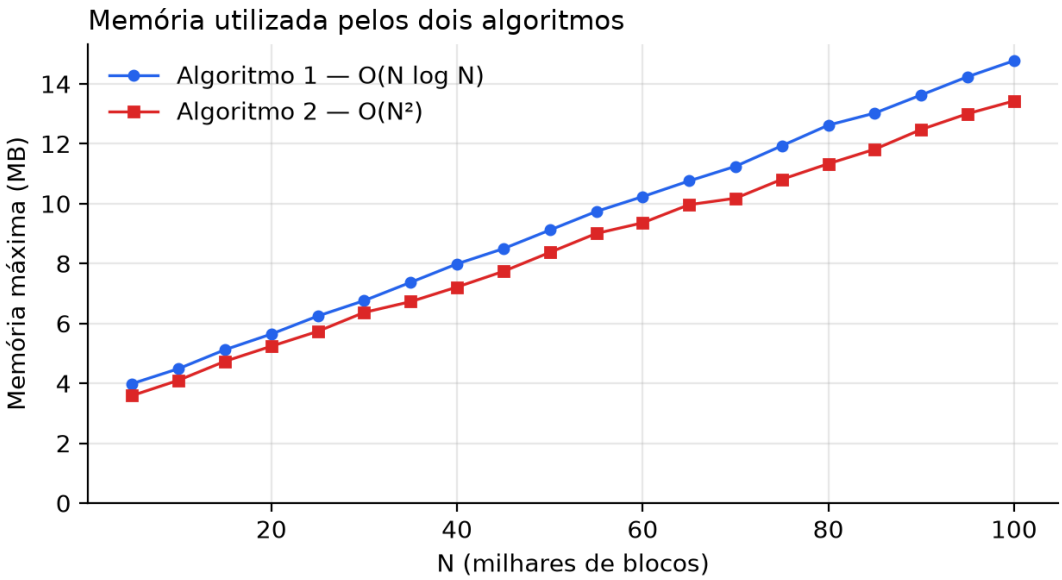


Figura 3 — Memória máxima residente dos dois algoritmos para as mesmas instâncias (N de 5 000 a 100 000).

Para comparar a memória, os dois algoritmos foram executados sobre as mesmas 20 instâncias. As duas retas são praticamente paralelas: em ambos os casos a memória cresce linearmente com N, cerca de 110 a 120 bytes por bloco, a partir de uma base fixa de cerca de 3,5 MB do próprio processo. Isto é o esperado, porque os dois algoritmos guardam as mesmas estruturas de tamanho O(N): as 3N orientações, um estado de programação dinâmica por orientação e a torre reconstruída. O algoritmo 1 usa apenas um pouco mais (cerca de 1,4 MB para N = 100 000) por causa do vetor de coordenadas comprimidas e da Fenwick Tree, que também são O(N). Assim, a complexidade espacial dos dois é O(N), e a vantagem do algoritmo 1 é obtida sem qualquer custo relevante de memória.

Conclusão

A substituição da procura linear pela melhor torre compatível por uma Fenwick Tree de máximos sobre coordenadas comprimidas reduziu a complexidade temporal de $O(N^2)$ para $O(N \log N)$, mantendo a complexidade espacial em $O(N)$. Os resultados experimentais confirmam a análise teórica: o tempo do algoritmo 1 acompanha a curva $c \cdot N \log N$ até 10 milhões de blocos, o do algoritmo 2 acompanha $c \cdot N^2$, e a memória dos dois cresce linearmente e de forma quase idêntica. Na prática, o algoritmo 1 resolve em cerca de 7 segundos instâncias que o algoritmo 2 levaria cerca de 17 dias a resolver.